

AsymptoticAnalysis

Incremental repository review

Native Wolfram Language comparison, Mathics compatibility,
new findings, and reproducible repair candidates

Repository	VladimirReshetnikov/Asymptotic
Review snapshot	7d1bc832895cc90a9b2a978b7b7684acab908bd2
Snapshot time	10 September 2026, 02:10:22 UTC
Local calendar date	9 September 2026, 19:10:22 PDT
Prior-review baseline	27 packages, three waves; 167 attributed ledger entries

Principal result. The portable test runner can forget a source change it already detected, then mark a changed-and-restored run as source-stable and exit successfully. This was reproduced by executing the byte-identical upstream Python runner against a controlled mock protocol peer. A candidate repair was checked against changed-source and unchanged-source controls.

Additional contributions. A nonfinite timeout defect, two Mathics allocation/search bottlenecks, a scale-dependent gap in affine eventual-domain proofs, and a local limit-default mismatch are analyzed separately. Exact reference models, a bounded affine-radius construction, candidate primitives, and unexecuted Wolfram/Mathics probes accompany the article.

Evidence boundary. No official Wolfram kernel or Mathics package run was completed in this review environment. Executed Python infrastructure tests, independent mathematical checks, source deductions, historical repository receipts, and unexecuted regression specifications are distinguished throughout.

Contents

1	Assessment and principal findings	1
2	Snapshot, scope, and evidence discipline	2
2.1	What was reviewed	2
2.2	Evidence labels	2
2.3	What was deliberately not rediscovered	3
3	Architecture and mathematical value	3
3.1	A real local coordinate and an explicit error object	3
3.2	Forward engines and inverse construction	4
3.3	Specialized scales should remain distinguishable	4
3.4	Three different kinds of validation	5
4	Comparison with native Wolfram Language	5
4.1	A fair comparison is not “series versus transseries”	5
4.2	Match the requested and achieved order	6
4.3	Native delegation does not imply analytic certification	7
5	N01: detected source drift can disappear from a successful receipt	7
5.1	Mechanism and impact	7
5.2	Executed reproduction against the original runner	8
5.3	Repair and remaining boundary	8
6	N02: nonfinite timeout values are accepted too late	9
7	N03: the Mathics coefficient adapter destroys sparsity	10
7.1	Source-level cost	10
7.2	A bounded candidate rather than an unsafe universal replacement	10
7.3	Acceptance obligations	11
8	N04: a first-match query materializes all matches	11
8.1	Why the obvious one-line repair is not established	11
8.2	Repair options and semantic constraints	11
9	N05: affine eventual-domain proofs depend on an arbitrary scale	12
9.1	The finite trial grid is not an affine completeness procedure	12
9.2	Predicted public symptom and why it is an availability defect	12
9.3	An exact constructive radius	13
9.4	How to integrate without weakening the existing branch contract	14

10 N06: the local limit adapter changes the default question	14
10.1 A conservative bounded policy	14
11 Mathics compatibility: what the current layer establishes	15
11.1 A versioned compatibility target, not a binary label	15
11.2 Do not equate expression construction with correct numerical use	16
11.3 What existing validation does and does not prove	16
12 Development priorities that follow from the new findings	17
12.1 First repair the evidence-producing infrastructure	17
12.2 Then close bounded adapter gaps with semantic laws	17
12.3 Profile the new adapters at the correct granularity	18
12.4 Handle the existing feature roadmap without claiming novelty	19
13 Artifacts, reproduction, and interpretation of results	19
13.1 What is included	19
13.2 Re-running the executed experiments	19
13.3 Applying the candidate infrastructure patch	20
13.4 Running the pending kernel probes	20
13.5 Limits of this audit	20
14 Conclusion	21
A Source map and audit scope	21
B Finding record format	22

1 Assessment and principal findings

The repository is more than a wrapper around ordinary power-series reversion. Its distinctive contribution is a family of explicit real asymptotic models: exact real powers, polynomial logarithmic coefficients, selected inverse branches, retained exact cores, several specialized scales, and structured remainder and provenance information. Native **Series** and **Asymptotic** results can also be retained, but deliberately have a different contract. The documented ambition to subsume those functions and **DiscreteAsymptotic** is a project requirement, not a description of completed coverage. A **DiscreteAsymptotic** backend is explicitly absent. [4, 3, 10]

The current Mathics effort is substantial. It addresses evaluator differences rather than merely checking whether a package file loads. Separate adapters handle control flow, associations, assumptions, held extraction, limited Taylor coefficients, selected special-function cores, and timing. These are meaningful improvements over a blanket “Mathics unsupported” assessment. Conversely, local compatibility definitions become part of the package’s mathematical and operational trust boundary. Their defaults, allocation behavior, and proof completeness need independent tests. [15, 17, 5]

ID	Priority / class	Finding	Evidence here
N01	High; validation integrity	A detected source mismatch is forgotten after restoration; final success can misstate source stability.	E1: original and candidate Python runners executed with a mock peer.
N02	Medium; runner input and interchange	<code>nan</code> and <code>inf</code> pass timeout validation, reach subprocess handling, and produce nonstandard numeric tokens in an initial JSON report.	E1: both arguments exercised; candidate rejects them before execution.
N03	Medium; availability / memory	Mathics sparse coefficient extraction constructs a full degree-sized coefficient list before discarding zeros.	S + E2; no large Mathics allocation attempted.
N04	Medium; avoidable work	The private <code>FirstPosition</code> computes and retains all matching positions. The obvious four-argument <code>Position</code> substitution is not supported by the inspected Mathics implementation.	S + E2; upstream Mathics source inspected.
N05	Medium; completeness / UX	Seven fixed radii cannot prove arbitrarily small, elementary affine deleted neighborhoods; equivalent germs become sensitive to coordinate scale.	S + E2; public package probes supplied but unrun.
N06	Medium; local semantic mismatch	Omitted direction in the Mathics <code>Limit</code> adapter becomes a lower-sided limit, unlike the current Wolfram two-sided real default.	S + exact mathematical witness; public reachability not established.

These are six different mechanisms, not six claims of wrong asymptotic coefficients. In particular, N03–N05 concern cost or refusal of valid requests; N06 is established at the private adapter boundary; and N01 concerns the reliability of a validation record. None demonstrates that a previously committed mathematical result is false or that a historical run actually suffered source drift.

The practical development order is to repair the validation runner, validate the new adapter laws in both entry forms, and only then widen Mathics coverage. More formula families will not compensate for ambiguous execution provenance or a primitive that changes the problem being solved.

2 Snapshot, scope, and evidence discipline

2.1 What was reviewed

The review pins the main-branch snapshot identified on the title page, including its modular source and generated-standalone loading design. The timestamp is on 10 September in UTC but on 9 September in the user’s Los Angeles time zone; there is no date inconsistency. Repository claims below refer to this commit, not an unversioned future `main`. [2, 5]

Source inspection concentrated on the core forward/inverse interfaces, native routing, inverse-branch validation, Fourier coefficient machinery, the new Mathics adapter family, the portable test runner, and its CI workflow. The user guide, Mathics compatibility and callable notes, and review-maintenance registers supplied the wider architectural and coverage context. This is not a claim that every source line, every vendored mathematical article, or all 27 review articles were independently re-audited. The source inventory in the archive records the actual scope.

The earlier review collection is unusually extensive: three waves contain 27 packages, with 167 attributed finding entries before overlap consolidation. The maintained implementation register and complete wave-3 intake were used as the duplication screen. Their current classifications were respected: a previously repaired defect was not reopened merely because an older report contains it. The latest review wave concerns snapshot 6687962; the Mathics compatibility file examined here is absent at that baseline. [6, 7, 8]

2.2 Evidence labels

E1: executed infrastructure. The upstream `validation/run_mathics_tests.py` was reconstructed from the pinned source and its Git blob identity was checked in memory against the connector’s reported identity. The bytes match. That original runner, and a narrowly modified candidate, were executed in temporary fixture repositories. The external “kernel” was a Python program implementing the runner’s documented output protocol and explicitly identifying itself as a mock. These experiments test runner logic, not Wolfram Language evaluation.

E2: executed independent reference checks. Python integer and rational arithmetic verified sparse/dense coefficient equivalence on small cases, the affine-radius construction, its coordinate-rescaling witnesses, a polynomial inverse oracle, and abstract first-match cost models. These checks validate mathematics and reference algorithms, not the evaluation of the supplied Wolfram Language candidate file.

S: source deduction. An implementation mechanism or direct mathematical consequence is visible in the pinned source. When an upstream interpreter feature matters, the Mathics 10.0.1 implementation or current official Wolfram documentation is cited. A source-deduced public outcome is still labeled as a prediction until a real kernel produces it.

U: unexecuted specification. The archive’s Wolfram Language probe script and candidate primitive definitions are prepared for fresh-kernel execution. They have not been validated in either mathematical runtime here.

The connected Wolfram evaluator failed before a useful evaluation, and the local environment did not provide Mathics. Consequently there is no fresh package-wide pass count, no measured Wolfram-versus-Mathics speed ratio, and no claim of final-source acceptance. Historical repository

receipts remain historical, scoped evidence. The Mathics compatibility guide itself distinguishes implemented adapters from complete, final-source acceptance. [15]

2.3 What was deliberately not rediscovered

The following adjacent topics already have owners in the review register. They are used only to explain contracts or distinguish the new mechanisms. [7, 8]

Existing owner	Existing topic	Boundary of this article's contribution
C01, C17; W3-04	Dense native-series conversion, native index range, eager native normal form	N03 is the distinct Mathics polynomial-to-coefficient-rules path, not the repaired native export issue.
P06, P08	Resource taxonomy and repeated symbolic work	N03/N04 identify concrete new adapter allocations; N02 exercises a particular Python timeout parser.
V01, V02	Release evidence and legacy aggregate-runner integrity	N01 is a loss of an already observed mismatch in the newer portable Python runner, not zero-test acceptance.
D01, D06; W3-01/02	Order, direction, and option-resolution policy	N06 concerns the local Mathics <code>Limit</code> default; it is not another report of public backend option parsing.
D02; W3-09	Capability/failure distinctions and conditioned native routing	N05 uses a purely real affine condition and the package backend; the fixed-radius proof search is the new cause.
C04, C05, C14, C15	Logarithmic error degree, assumptions, equal weights, varying-parameter composition	The current fixes and residual boundaries remain the baseline; no duplicate defect claims are made.
X01–X12	Broader scales, integration, complex sectors, uniformity, systems, and solver adapters	These remain recorded development directions, not newly discovered missing features.

3 Architecture and mathematical value

3.1 A real local coordinate and an explicit error object

A useful way to understand the core is to separate a coordinate, a finite jet, and an error class. Near a finite endpoint the coordinate has the form

$$x = x_0 + \sigma u, \quad u \downarrow 0, \quad \sigma \in \{-1, 1\},$$

while infinite endpoints use a signed reciprocal coordinate. In the ordinary power-log case, a finite model is naturally written

$$J(u) = \sum_{\alpha < h} u^\alpha P_\alpha(\log u), \quad f(u) - J(u) = \mathcal{O}\left(u^\beta (1 + |\log u|)^d\right).$$

The supported exact exponents need not lie on one rational lattice. Terms at the same mathematical weight must be combined before deciding support, sign, or truncation. The repository’s core and guide encode this distinction between a finite expression and its retained remainder data. [9, 4]

The leading power dominates every fixed logarithmic degree. This makes a source-power grading useful without pretending that logarithmic coefficients are constants. A support containing positive generators $\lambda_1, \dots, \lambda_m$ is locally finite: a bound $k \cdot \lambda < H$ gives $0 \leq k_i < H/\lambda_i$ for every nonnegative integer index. That elementary observation explains why finite target cutoffs can support exact irrational weights. It is not a claim that index enumeration is always inexpensive.

The result object, **GeneralizedSeries**, is therefore more informative than its printed finite approximation. It can retain a source model, a chart, assumptions, branch data, an error description, and a refinement recipe. **Normal[s]** and numerical substitution into the stored expression do not by themselves consume or certify all that information. This distinction is already documented and is not a new complaint about unchecked numerical application. [9, 4, 7]

3.2 Forward engines and inverse construction

The forward path combines explicit algebraic operations with admitted Taylor, Laurent, or Puiseux information and specialized adapters. The inverse path recognizes leading monomial and logarithmic/exponential structures, uses Lagrange-type or Newton/grouped mechanisms where admitted, and transports the result to the requested source and target coordinates. Inverse expressions embedded in a larger calculation first require a selected real germ; a finite list of branch candidates is not treated as a completeness theorem. [9, 11, 12, 13]

This is valuable for expressions whose natural output is not an ordinary integer-power series. For example, consider the mathematical model

$$f(u) = u \left(1 + u^{\sqrt{2}}(1 + \log u) + u^{\sqrt{3}} \right), \quad u > 0.$$

Its derivative tends to 1, so an increasing inverse germ exists near zero. Writing $f(u) = u + \eta(u)$ and solving $u = y - \eta(u)$ gives the first correction

$$f^{-1}(y) = y - y^{1+\sqrt{2}}(1 + \log y) - y^{1+\sqrt{3}} + \mathcal{O}\left(y^{1+2\sqrt{2}}(1 + |\log y|)^2\right).$$

Indeed, $\eta(y) = \mathcal{O}(y^{1+\sqrt{2}}(1 + |\log y|))$ and $\eta'(y) = \mathcal{O}(y^{\sqrt{2}}(1 + |\log y|))$; the mean-value estimate for replacing $\eta(u)$ by $\eta(y)$ yields the stated remainder once $u/y \rightarrow 1$. This is an illustrative target and independent derivation, not a recorded kernel output. It shows why explicit weight and logarithmic-error bookkeeping is useful even when a native system can also manipulate parts of the expression.

3.3 Specialized scales should remain distinguishable

The module structure includes logarithmic hierarchies, retained exact inverse cores, flat exponential sectors, Fourier-polynomial logarithmic coefficients, and selected Gamma/Barnes and other special-function families. Their order parameters need not mean the same thing: one cutoff may grade the positive local source variable, another the correction multiplying an exact prefactor, and another separate carrier components. An exact prefactor is not an omitted term and should not be expanded merely to make the output resemble a plain polynomial. [5, 4, 13, 14]

The Fourier coefficient module, for example, stores finitely many exact frequencies together with polynomial amplitudes in $\log u$. It checks conjugacy for real-valued coefficients and treats a frequency budget as a resource constraint, not as permission to discard inconvenient modes. Its sorted source-weight multiplication and binary powering already exist. A proposal simply to add those named algorithms would not be incremental. [14, 7]

3.4 Three different kinds of validation

A formal residual identity checks the algebra of an admitted finite model. A high-precision numerical residual checks selected sample points and numerical conditioning. An inverse certificate additionally needs a proved interval, monotonicity or derivative information, and bounds applying to the function being certified. These claims are not interchangeable. The public API’s separate residual, numerical-check, and certificate operations is a sound architectural direction, even though the existing register records important remaining scope and precision issues. [4, 7]

This review adds a fourth, operational prerequisite: the source identity of a validation run must itself be reliable. A correct finite residual produced from one source state does not establish that a receipt describes a different source state. N01 concerns this prerequisite rather than changing the mathematics of a residual or certificate.

4 Comparison with native Wolfram Language

4.1 A fair comparison is not “series versus transseries”

Native **Series** already handles selected negative and fractional powers, logarithms, infinite endpoints, and successive expansions in multiple variables. Its options include how unrecognized functions are treated analytically. Native functionality should therefore not be described as restricted to ordinary Taylor polynomials. [31]

A **SeriesData** object provides a coefficient list on a rational-power index lattice, but native results can keep symbolic powers or rapidly varying factors outside that object. The representational advantage here is the package’s explicit, unified handling of admitted real weights and error metadata—not an assertion that Wolfram Language cannot represent irrational powers or exponential prefactors at all. [32]

InverseSeries performs series reversion and is not limited to a nonzero linear leading term: the official examples include a quadratic-leading series. **AsymptoticSolve** also supplies asymptotic solutions to equations and systems, with real-valued solution selection available. Those are relevant comparators for inverse work; comparing only with **Series** understates the native baseline. [33, 35]

Native **Asymptotic** admits more than explicit elementary expressions, including integral and differential-equation forms, finite or infinite real or complex centers, and problem-inferred scales. Its documented asymptotic approximation contract is not identical to the package’s explicit **PowerLogRemainder** representation. A wrapper may preserve a native result without having converted its evidence into a package analytic contract. [34, 10]

Task	Native baseline	Package position at the reviewed snapshot
Ordinary local expansion	Mature Series / SeriesData algebra.	Custom exact real power-log path plus explicit and selected automatic native delegation.
Irrational and logarithmic grading	Symbolic factors and slow coefficient functions can occur in native representations.	Admitted real weights, logarithmic coefficient polynomials, and explicit error grades share one custom model.

Task	Native baseline	Package position at the reviewed snapshot
Inverse expansions	InverseSeries and equation-based AsymptoticSolve ; selected native special-function identities.	Specialized real inverse constructors, branch provenance, retained cores, and model-specific residual checks.
Exponential and oscillatory forms	Native expressions can retain rapid factors; Asymptotic infers problem-dependent scales.	Explicit flat-sector and finite Fourier-logarithmic models, within their documented restrictions.
Complex and multivariate problems	Native expansion/solver interfaces include such forms.	Native delegation can preserve eligible results; the custom analytic model is not a general complex or multivariate calculus.
Discrete asymptotics	DiscreteAsymptotic is a separate native interface.	No implemented backend of that name; the guide treats complete subsumption as unfinished.
Remainder and provenance operations	Formal/native objects and native asymptotic contracts.	Dedicated error metadata and refinement/certification interfaces for admitted custom models; native objects remain separate contracts.
Mathics portability	The official Wolfram implementation is not present in Mathics simply because a symbol name exists.	A growing private compatibility layer; implementation and validation remain operation-, branch-, and runtime-specific.

Sources for the comparison: package guide and native dispatch [4, 10]; native documentation [31, 32, 33, 34, 35, 36]; Mathics boundary [15].

4.2 Match the requested and achieved order

For the branch of $x + x^2 = y$ through zero,

$$x(y) = \frac{\sqrt{1+4y}-1}{2} = y - y^2 + 2y^3 - 5y^4 + \mathcal{O}(y^5).$$

The guide’s package request with target cutoff 5 retains powers strictly below 5. An ordinary native series requested through degree 4 is the matching comparison, not necessarily one requested with the same literal integer 5. [4, 31]

A precise native reversion specimen can start from an explicitly structured input, avoiding assumptions about whether a particular polynomial expansion is returned as a bare polynomial or a series object:

```
InverseSeries[SeriesData[x, 0, {1, 1}, 1, 5, 1], y]
```

This is an unexecuted native comparison request. Its expected coefficients through degree four follow from the displayed exact inverse. The independent Python oracle checked the same Catalan-coefficient identity through degree 12, not the execution of **InverseSeries**.

For benchmarking, record the achieved exponent or error scale, endpoint, branch, parameter conditions, and whether a retained prefactor is expanded. Only compare timings after those objects match. The old register already asks for matched precision and cold/warm measurements; the new

Mathics-specific addition is to record evaluator overhead, adapter selections, and refusal causes separately from mathematical coefficient work. [7, 15]

4.3 Native delegation does not imply analytic certification

The source explicitly rejects some custom analytic operations on a native result and directs callers to its retained native value. That separation avoids inventing an analytic remainder merely because the outer head is `GeneralizedSeries`. Native backend selection, native outcome status, and package failure information are retained as distinct metadata. [10]

The user-facing implication is simple: success under a native backend should be read as success under that backend's contract. It is not evidence that the custom real branch analysis, remainder transport, or inverse certificate was performed. Likewise, a mathematically supported native operation on the Wolfram kernel may remain unresolved on Mathics. This limitation follows from the actual engine selected, not from a contradiction in the wrapper's result type. [4, 15]

5 N01: detected source drift can disappear from a successful receipt

5.1 Mechanism and impact

The portable runner is designed carefully in several respects. Each selected case runs in a fresh process; complete, matching output records are required; process exit status is reconciled with the reported outcome; the suite text is copied to a temporary file; and reports are written through an atomic replacement. The process timeout terminates the process group or owned process tree rather than indiscriminately killing other kernels. These mechanisms are worth preserving. [27]

The source-stability check, however, is not cumulative. At startup the runner records `before = fingerprints(source)`. Each call to its nested `write_report` obtains a new snapshot, then writes `SourcesUnchangedDuringRun: before == after`. The final return code uses that final boolean. Although intermediate reports are written after individual cases, their earlier source-mismatch result is not retained in memory. [27]

Let H_0 be the initial fingerprint map and H_i the map observed after case i . The current final acceptance predicate is essentially

$$\text{all cases successful} \quad \wedge \quad H_m = H_0.$$

The intended sampled stability predicate is stronger:

$$\text{all cases successful} \quad \wedge \quad \bigwedge_{i=1}^m (H_i = H_0).$$

These are different even when every fingerprint operation itself works correctly. Restoration is not evidence that earlier cases used the initial sources.

This is not merely the usual limitation that a file can change and change back between two observations. Here an observation actually detects the mismatch, and a later observation erases its effect on acceptance. A developer switching branches, rebuilding generated files, or temporarily editing modules during a long run could encounter this sequence accidentally. The fixture deliberately makes it deterministic; it does not establish that any published repository receipt was affected.

5.2 Executed reproduction against the original runner

The supplied `upstream/run_mathics_tests.py` is the retrieved runner, with its Git blob identity checked against the pinned upstream file. The fixture creates a disposable repository-shaped directory containing a suite with two IDs and a modular source with a sibling probe module. The external executable is a small Python protocol peer, explicitly identifying itself as a mock rather than a mathematical kernel.

On the first case, the peer changes the sibling module and emits a successful protocol record. The runner then records a source mismatch. On the second case, the peer reads that interim report and the changed module, records both observations, restores the initial module bytes, and emits another success. The actual upstream runner performs its own fingerprinting, process launch, parsing, report writing, and final exit decision throughout this experiment.

Scenario	Original exit	Candidate exit	Final stability, original / candidate
Sources never changed	0	0	True / True
Change persists through completion	1	1	False / False
Change detected, then restored	0	1	True / False

Table 4: Executed Python-runner results. Mock protocol successes are not Wolfram or Mathics package test successes. Raw records are in `evidence/runner_experiments.json`.

In the restored-source case, the fixture also confirms that the interim report said the sources had changed and that the second peer read the changed module before restoration. Thus the result is not inferred solely from an exit code. The persistent-change control shows that the original runner already detects ordinary un-restored edits; the defect is specifically loss of historical invalidity.

5.3 Repair and remaining boundary

The candidate adds a monotone flag. Once a mismatch has been observed, it remains true for the lifetime of the run:

```
source_drift_observed = False
source_drift_observations = []

# Inside write_report:
nonlocal source_drift_observed
if before != after:
    source_drift_observed = True
    # Record changed paths and the number of completed cases.

# In the report:
"SourcesUnchangedDuringRun": not source_drift_observed

# The original final return-code test now consumes this cumulative value.
```

The implementation preserves a concise history of changed paths and adds `SourceCheckScope: "Boundary samples, not continuous monitoring"`. The exact patch and guarded patch generator are included. The original, unchanged-source behavior and persistent-change refusal remain intact in the executed controls.

A latch is necessary but not sufficient for stronger provenance. It cannot observe edits that occur

entirely between samples. The next step is to freeze all executable package inputs, not only the suite. For a modular entry point, copy its complete sibling module set into a private directory preserving the loader’s relative paths. Invoke that entry point, retain the original-to-frozen path map, and treat changes to the original source tree as a separate invalidity condition. For the standalone entry, freeze the complete single file. Any capture script and launcher whose behavior affects the receipt should belong to the same frozen input set.

This stronger design should not silently copy the entire repository: only the resolved executable dependency closure is needed. Nor should it claim that ordinary read-only permissions prevent every possible mutation by the owning process. State the threat and concurrency model. A reproducibility receipt should distinguish “initial and final bytes agree,” “all sampled checks agree,” and “execution used a frozen source snapshot.”

The new finding is adjacent to existing validation recommendations, but it is not the old aggregate runner’s zero-test acceptance problem, nor a request to add CI that already exists. It identifies an executed false-positive acceptance path in the newer portable runner. [7, 28]

6 N02: nonfinite timeout values are accepted too late

The command-line parser reads `-timeout` using Python’s `float`. The subsequent guard checks only `args.timeout <= 0`. Positive infinity and NaN pass that check: infinity is positive, and an ordered comparison with NaN is false. Both therefore reach report creation and subprocess timeout handling instead of producing a normal command-line diagnostic. [27]

The actual upstream runner was exercised with `nan`, `inf`, `0`, and `-1`. In this Linux/Python environment, the two nonfinite arguments resulted in unsuccessful execution with Python-level diagnostics; the initial report remained incomplete. Its timeout field contained the literal `NaN` or `Infinity`, respectively. The latter are accepted by Python’s permissive JSON writer, but are not numbers in the JSON grammar. Interoperable JSON does not permit either token. [40]

Input	Original runner, executed	Candidate runner, executed
<code>nan</code>	Exit 1; incomplete initial report contains <code>NaN</code> ; exception-path diagnostics.	Exit 2; command-line refusal; no report created.
<code>inf</code>	Exit 1; incomplete initial report contains <code>Infinity</code> ; exception-path diagnostics.	Exit 2; command-line refusal; no report created.
<code>0</code> , <code>-1</code>	Exit 2; correctly rejected.	Exit 2; correctly rejected.

Table 5: Timeout characterization with a disposable mock protocol peer. Absence of a peer marker does not prove that `Popen` was never called.

The candidate requires a finite, positive number before computing fingerprints, writing a report, or invoking a subprocess:

```
if not math.isfinite(args.timeout) or args.timeout <= 0:
    parser.error("--timeout must be finite and positive")
```

This small change is included in the same patch as N01. A distinct follow-up would make JSON serialization explicitly strict, with `allow_nan=False`, and provide a deliberate diagnostic if another report field unexpectedly contains a nonfinite number. That serializer change is recommended, not included in the tested patch; applying it without deciding how to report a serialization failure would merely move the exception.

The issue is a validation and interchange defect, not evidence of an erroneous asymptotic formula. There is also no reason to infer that positive infinity provides a reliable “unlimited” mode across operating systems. An unlimited mode, if desired, should have an explicit representation and a separately documented execution policy rather than relying on exceptional floating-point values.

7 N03: the Mathics coefficient adapter destroys sparsity

7.1 Source-level cost

The Mathics-specific univariate `CoefficientRules` adapter obtains a dense `CoefficientList`, pairs each entry with its exponent, removes zero entries, and reverses the order. This implements a sparse-looking output by first materializing all intervening coefficients. [18] For

$$P_N(L) = 1 + L^N,$$

the input and desired coefficient-rule output have two nonzero monomials, whereas the intermediate list has $N + 1$ positions. The relevant native coefficient-list interface explicitly fills interior zeros and sizes the array by degree. [37]

In terms of support size s and maximum degree N , a representation with $s = 2$ has been converted to an $\Omega(N)$ allocation and traversal. A subsequent support limit cannot recover that work. This matters particularly for power-log algorithms: a high logarithmic degree may be encoded in a short expression even when only a few source-power blocks are requested.

The polynomial-domain helper `mathicsPolynomialFunctionDomain` also uses a dense `CoefficientList` to establish that every polynomial coefficient is an exact real constant. Implicit zero coefficients need not be materialized to establish that theorem. The sparse coefficient iterator could therefore serve both extraction and bounded real-domain admission. [21]

There are other dense coefficient-list consumers in the broader package, including Fourier polynomial normalization. Their existence should be included when profiling an end-to-end request, but is not claimed here as another new finding. In particular, the public request may encounter an earlier normalization or proof limit before reaching the new adapter. This review does not claim a measured public out-of-memory result. [14, 7]

7.2 A bounded candidate rather than an unsafe universal replacement

The accompanying `SparseMonomialRules` primitive admits an *already expanded* sum of monomials in one symbol. Each factor must be independent of that symbol, the symbol itself, or a nonnegative integer power of it. It extracts exponents, groups equal integer exponents, combines coefficients, removes structural zeros, and sorts the retained support. It does not call `Expand` or allocate an array indexed by the largest exponent.

For this admitted input syntax, the work is driven by the number of input factors and monomials, plus support grouping and sorting, rather than by the largest gap in the exponent set. Exact symbolic coefficient simplification is not assumed free. The candidate does not promise optimal complexity for Mathics’ list-building primitives, but it removes the degree-sized intermediate array on this path.

The restriction is essential. Calling an unrestricted `Expand` on $(1+L)^N$ can itself create $N + 1$ genuinely nonzero monomials; no sparse representation change eliminates that output size. Conversely, silently treating an unexpanded variable-dependent factor as a coefficient would be wrong. The candidate refuses such expressions and accepts an explicit input monomial budget. A production adapter could use this fast path and retain a separately budgeted fallback for small dense polynomials.

The independent exact Python checks compare sparse aggregation with a dense oracle in 325 bounded cases. A separate symbolic degree- 10^9 specimen records two nonzero output terms and the 1,000,000,001 positions that the source algorithm would require. It does *not* allocate the latter array and is not a Mathics memory benchmark. Wolfram/Mathics execution of the supplied primitive remains pending.

7.3 Acceptance obligations

A production change should be accepted only after testing zero and constant polynomials, repeated exponents, coefficient cancellation, exact irrational coefficients independent of the polynomial variable, rejected Laurent and nonpolynomial terms, and low-degree fallback equivalence. Test fixed support with increasing maximum degree separately from dense support with increasing degree. A real-domain consumer must check every retained coefficient and must not infer the reality of a symbolic coefficient from the fact that it is free of the expansion variable.

This mechanism is distinct from the repaired optional `SeriesData` export allocation: the latter concerns rational source-exponent lattices and a native view of an existing series. N03 concerns coefficient algebra and Mathics branch admission before that export. Reusing the same word “sparse” for both problems should not hide their different execution paths. [7, 18]

8 N04: a first-match query materializes all matches

The local `FirstPosition` adapter appropriately keeps a potentially stateful default held until no match exists. Its search helper nevertheless executes `Position[expr, pattern, levels, Heads -> heads]` without a result-count bound, retains the entire result, and then takes `First`. A tree with m matching nodes therefore produces m stored position vectors when the consumer needs only one. Even if the desired node occurs immediately, the full search is performed. [17]

The avoidable output storage is at least proportional to the total length of all matching position vectors, not simply the depth of the first answer. A flat list with n matching entries gives a transparent example: the current approach stores n one-component positions; an early-exit search needs one. The accompanying flat-list reference models make this counting distinction explicit for four sizes through 100,000. They are counting models, not measured kernel timing comparisons.

8.1 Why the obvious one-line repair is not established

In the Wolfram interface, `Position` can accept a maximum result count as a fourth positional argument. However, the inspected Mathics 10.0.1 implementation declares the expression, pattern, optional level specification, and options, then traverses the tree and appends every match. It does not implement the corresponding maximum-count positional overload. [38, 29]

Consequently, replacing the call with `Position[expr, pattern, levels, 1, Heads -> heads]` is not a verified Mathics fix. It could simply leave the expression unresolved. A compatibility review should inspect the receiving interpreter rather than propose a native Wolfram spelling on the assumption that it is portable.

8.2 Repair options and semantic constraints

The cleanest general repair is an upstream bounded `Position` operation that stops traversal as soon as enough matches have been found, together with an implementation-level regression proving bounded traversal. A package-local repair can instead provide narrow fast paths for the actual level

patterns used by its consumers and conservatively delegate remaining cases. A complete private walker is possible but should be justified by the maintenance cost.

The compatibility tests must preserve traversal order, root matching, levels specified by depth and negative depth, inclusion or exclusion of heads, and the lazy default. Do not trade a memory issue for a wrong position. Likewise, do not broaden a small internal adapter into an undocumented replacement for all native pattern-search forms. This article does not distribute a speculative walker and label it production-ready; the supplied probe exposes the current helper for later runtime characterization.

This is a concrete new internal Mathics search cost, adjacent to but more specific than the earlier general profiling recommendation. It is not a claim that every package operation is dominated by `FirstPosition`, or that stateful pattern tests have a portable, guaranteed evaluation-count contract. [7, 17]

9 N05: affine eventual-domain proofs depend on an arbitrary scale

9.1 The finite trial grid is not an affine completeness procedure

The Mathics branch adapter contains a useful exact interval test. For an affine expression, it examines the two endpoint values and can prove a relation throughout the intervening deleted interval. This is sound: each interior value is a strict convex combination of the endpoints. The problem is not that test, but how a radius is chosen when proving an eventual condition. [21, 16]

The override of `inverseFunctionEventually` first tries symbolic simplification under $u > 0$, then tests radii 2^{-j} for $j = 0, \dots, 6$. If no trial works, it invokes the older generic proof path. That path asks for an existential radius followed by a universal neighborhood condition using `Resolve`. General quantifier elimination is not supplied by the Mathics compatibility layer. [21, 11, 15]

Consider the elementary condition

$$0 < u < \varepsilon, \quad 0 < \varepsilon < \frac{1}{64}.$$

It is eventually true as $u \rightarrow 0^+$, but no radius in the seven-element grid lies inside that neighborhood. The statement $u < \varepsilon$ is not true for all $u > 0$, either. Thus neither unrestricted simplification under $u > 0$ nor any of the seven explicit interval tests supplies the required proof. The remaining route depends on the unavailable general quantifier solver.

The exact family in the reference checks uses $\varepsilon = 2^{-p}$ for $p = 1, \dots, 160$. Six members have an adequate radius in the grid and 154 do not. All 160 admit a radius constructed directly from their coefficients. These counts describe the independent exact model of the grid and the proposed radius rule; they are *not* 160 executed package requests.

9.2 Predicted public symptom and why it is an availability defect

The forward public boundary calls `inverseFunctionEventually` to admit an approach condition. Its refusal tag is `IncompatibleTargetCondition`. The direct inverse boundary uses the same eventual proof mechanism and an analogous source-condition refusal. [11] A focused request for later runtime reproduction is:

```
AsymptoticExpansion[
  ConditionalExpression[x + x^2, 0 < x < 2^-100],
  {x, 0, 4}, "Backend" -> "Package"]
```

The same request with upper endpoint 1 is a useful control. Both describe the same polynomial germ at zero. Its exact forward expression is unchanged; its local inverse is

$$g(y) = \frac{\sqrt{1+4y} - 1}{2} = y - y^2 + 2y^3 - 5y^4 + \mathcal{O}(y^5).$$

For either positive upper endpoint, $1 + 2x > 0$ on a sufficiently small right neighborhood. There is no mathematical branch obstruction.

The source predicts a proof-availability failure for the tiny neighborhood in the documented Mathics regime, not a wrong finite expansion or an unsound certificate. This review has not executed that public request. It is included in `tests/ProbeNewReview.wl` so that an actual kernel run can confirm the complete dispatch path, any intervening simplification, and the exact outcome.

The distinction is important for UX. An unresolved proof is not a disproved condition. The current public failure wording can make a valid small neighborhood appear incompatible. Within a documented bounded affine class, shrinking a positive neighborhood should not turn a successful germ into a failure merely because the new endpoint lies below a fixed trial grid.

9.3 An exact constructive radius

The following elementary lemma eliminates the arbitrary grid for rational affine conjunctions. It uses only exact arithmetic and sign tests.

Lemma 9.1 (A sign-preserving affine neighborhood). *Let $a, b \in \mathbb{Q}$, and let $L(u) = a + bu$. If $a \neq 0$ and $b \neq 0$, define $r = \min(1, |a|/(2|b|))$. Then $L(u)$ has the sign of a for every $0 < u < r$. If $b = 0$, its sign is constant. If $a = 0$, its sign for $u > 0$ is the sign of b , with $L \equiv 0$ when both vanish.*

Proof. In the nonzero case, $|bu| < |a|/2$, so $L(u)$ cannot reach or cross zero and has the sign of a . The two remaining cases follow by direct substitution. $\square$

Proposition 9.2 (Finite affine conjunctions). *For a finite set of rational affine clauses*

$$a_i + b_i u \bowtie_i 0, \quad \bowtie_i \in \{<, \leq, >, \geq, =, \neq\},$$

the local signs in the lemma decide eventual truth of every clause. If all clauses are eventually true, the minimum of their positive sign-preserving radii, with a cap of 1, proves the entire conjunction on a positive deleted interval.

Proof. A strict positive or negative relation requires the corresponding nonzero local sign. A non-strict relation also permits zero; equality requires the identically zero affine polynomial; inequality requires a nonzero local sign. These conditions are necessary and sufficient in the neighborhood from the lemma. A finite minimum of positive rational numbers remains positive and simultaneously preserves every established sign. $\square$

For $u < \varepsilon$, write $\varepsilon - u > 0$. The construction gives $r = \min(1, \varepsilon/2)$, with no lower scale threshold. Bounds need not be optimal; they only need to be positive and proved. A positive rational change of coordinate alters the radius but does not destroy the existence of one.

The supplied `AffineRadius` primitive implements precisely this rational clause API. It is not a parser for arbitrary Wolfram logical expressions and not a replacement for `Reduce`. Unsupported symbolic coefficients or relation forms receive a distinct failure. A production integration should normalize admitted ordered operands only after establishing their realness, then call the radius constructor; it must not cancel a shared unknown complex offset in an inequality.

In addition to the 160-member family, independent exact tests examine 600 random finite rational conjunctions. The construction proves 124 of them, and an endpoint oracle checks the whole proposed

interval for each proved case. Twenty-five positive coordinate rescalings are also checked. This is evidence for the mathematical construction, not execution of its Wolfram Language translation.

9.4 How to integrate without weakening the existing branch contract

Use the constructive radius as another bounded proof path before general quantifier elimination. Retain its explicit radius and normalized clauses as proof metadata. A failed rational sign test may establish local falsity for the admitted clause; a failed parse or unproved symbolic sign must remain inconclusive. Do not convert either into global nonexistence of an inverse.

The branch validator must still check the function’s real domain, target limit, approach side, and local derivative sign. The new radius proves only a neighborhood condition. The existing distinction between a condition radius and a quantitative asymptotic-error threshold remains valid and should not be blurred. [12, 16]

This proposal is deliberately narrower than the earlier parameter-case and branch-discovery roadmaps. It closes a concrete scale-dependent hole in the new Mathics affine proof path, using a complete elementary procedure for that limited input class.

10 N06: the local limit adapter changes the default question

The current Wolfram `Limit` interface defaults to a real two-sided approach at a finite real point, expressed as `Direction -> Reals`. Explicit right and left approaches correspond to `-1` and `1`, respectively. The Mathics compatibility adapter instead declares `Direction -> Automatic` and maps that default to `1`, selecting only the lower side. [39, 20]

The mathematical distinction is witnessed without a computer algebra system:

$$h(u) = \frac{u}{|u|}, \quad \lim_{u \rightarrow 0^-} h(u) = -1, \quad \lim_{u \rightarrow 0^+} h(u) = 1.$$

There is no two-sided real limit. A helper that returns the lower-sided value for an omitted direction has answered a different question. This is an established mismatch in the inspected private adapter’s contract; the particular Mathics evaluation of this witness was not run here.

Crucially, this does *not* establish a current wrong public inverse result. The inspected branch-validation call already specifies `Direction -> -1` explicitly when taking a limit in its positive local coordinate. That call is not affected by the omitted-direction default. A consumer audit is needed before assigning public reachability or a higher severity. [12]

10.1 A conservative bounded policy

For a finite endpoint, an adapter can compute the two explicit one-sided limits. It should accept a common value only after establishing equality and resolving the computations. If the results differ or remain unresolved, return a structured “two-sided limit not established” result. Distinguish this from a proof that the limit does not exist: inability to establish equality is not disequality.

The accompanying `ConservativeRealLimit` candidate accepts structurally identical resolved values, deliberately sacrificing some completeness. It passes explicit supported one-sided directions through, and treats the two signed infinite endpoints using the corresponding approach from the real axis. It does not claim complex-path limits, general directional vectors, or all native option behavior.

This is a demonstration primitive, not an installed override. Integrating its failure result requires auditing every private consumer’s resolvedness checks: a new `Failure` object must not accidentally be treated as a finite limit or passed to a branch proof. Consumers that truly need $u \rightarrow 0^+$ should

say so explicitly and avoid a redundant two-sided computation. The focused tests should include continuous controls, jumps, opposite signed infinities, unevaluated limits, equal-but-not-structurally-identical results, and explicit one-sided requests.

11 Mathics compatibility: what the current layer establishes

11.1 A versioned compatibility target, not a binary label

The compatibility guide identifies Mathics3 10.0.1, scanner 10.0.1, SymPy 1.14.0, and Python 3.11 as its development environment. Those versions define the relevant target here; this article does not assert that they are the newest available releases. The guide explicitly leaves consolidated final-source acceptance pending and separates focused examples from full public-interface coverage. [15]

The central architectural choice is sensible: compile selected package calls against a temporary, isolated compatibility context, remove that context from the public search path after loading, and leave ordinary Wolfram execution on its original path. Some missing public input names are established as inert system symbols; their existence is not an implementation of the associated function. Reload behavior is separately handled because Mathics can evaluate old definitions while new left-hand sides are installed. [17, 5, 9]

Area	Inspected support	Remaining interpretation / obligation
Control flow and associations	Scoped return tags, lookup and membership helpers, local update operations, held defaults.	Preserve nested scopes and callback effects; a helper is not full emulation of every native overload. [17]
Held callables	Wrapped extraction preserves selected parts while the interpreter performs actual function application.	Test assigned formal symbols, nesting, slot arity and capture avoidance; do not replace this with textual substitution. [22, 16]
Realness and signs	Bounded exact consequences of conjunctions; ordered predicates screened for real operands.	No general quantifier elimination or complete algebraic decision procedure. Unproved signs must remain unresolved. [19, 20]
Polynomial branches	Exact-real polynomial domain rule and affine interval / convexity proofs.	N03's degree allocation and N05's fixed-radius completeness gap need targeted repairs. [21]
Taylor and special functions	Finite defining series for selected zero-centered monomial arguments; selected Stirling and exact-core adapters.	Not arbitrary special-function continuation, simultaneous parameter asymptotics, or unlimited defining-series expansion. [23, 24]
Exact algebra	Some unavailable normalization retains exact symbolic forms instead of approximating them.	Exact retention is good, but does not supply every equivalence proof or algebraic canonical form. [17]

Area	Inspected support	Remaining interpretation / obligation
Lambert cores	Selected principal-branch construction sites use the one-argument form of <code>ProductLog</code> .	Nonprincipal numerical evaluation remains a documented limitation; symbolic construction alone is not numerical compatibility. [25, 15]
Native delegation	Requests can preserve the interpreter’s native output separately from an analytic package result.	An inert <code>Asymptotic</code> or unsupported native option does not satisfy the Wolfram coverage target. [10, 15]
Numerical checks and certificates	The guide records selected smoke checks and exact rational certificate examples.	No new numerical-accuracy or certificate acceptance is established in this review. [15]
Formatting and performance budgets	Runtime-specific formatting and private proof-time allowances; larger iteration budget is an explicit session setting.	Output appearance, evaluator iterations, process deadline and mathematical support budgets remain distinct. [15, 26]

11.2 Do not equate expression construction with correct numerical use

The principal Lambert workaround is a useful example of the distinction. Selected constructors rewrite the principal branch to its one-argument `ProductLog` spelling before the problematic two-argument numerical path can see numeric data. The inspected `Mathics` source connects `ProductLog` to the numerical Lambert function without a corresponding explicit branch/argument-order adapter in that class. The repository consequently limits its numerical claim and documents nonprincipal evaluation as unsupported. [25, 30, 15]

This is not presented as another newly discovered defect: the limitation is already documented in the new `Mathics` material. The development implication is that compatibility tests should separate a symbolic constructor, its `Normal` expression, numerical substitution of that expression, and a package numerical-check function. Success at the first stage cannot stand in for the others. A future private numerical wrapper should preserve branch identity and precision and be checked against an independent defining equation and branch inequalities, not only a decimal value.

Similarly, the documented `s[value]` operation is ordinary substitution into the finite expression, not a certified evaluation. The earlier review register already discusses a separate checked evaluator. This article does not repeat that proposal as a fresh API bug. [4, 7]

11.3 What existing validation does and does not prove

The repository has a `Mathics` CI matrix with modular and standalone entries, five grouped selections, Python 3.11, fresh-process portable tests, and a per-case timeout. It also has standalone-source freshness checking. Thus a recommendation merely to “add `Mathics` CI” would be stale. [28]

The compatibility guide additionally describes native definition comparisons and historical Wolfram preservation runs. Their value is that they check concrete recorded properties against specific controls. Definition equality, a selected behavior probe, and a historical full run are different evidence populations; none establishes a new complete run at this review’s snapshot. N01 concerns the portable runner’s cumulative source check and must not be imputed automatically to a different

capture/comparison tool. [15, 27]

No working Wolfram or Mathics runtime was obtained during this review. The attempted Wolfram connector calls failed at the service/network layer, not with mathematical errors from the package. The current source was inspected through the GitHub connector; the complete repository was not executed or exhaustively line-audited. The experimental claims in this article are limited to the exact Python runner and independent reference tests actually supplied and run.

12 Development priorities that follow from the new findings

12.1 First repair the evidence-producing infrastructure

N01 should precede relying on new portable acceptance receipts: it is small, reproduced, and affects how results are attributed to sources. Integrate the cumulative-invalidity patch, retain the restored-source fixture, and require that the final exit code agree with the latched receipt. Add N02's finite positive timeout validation in the same infrastructure change. These repairs do not require a mathematical kernel to test.

Next, freeze the complete executable source closure and record the actual frozen entry point passed to the kernel. The current runner's command metadata starts from the unfrozen command while execution substitutes the frozen suite; that is understandable implementation history, but a future receipt should make original and executed paths explicit rather than force a reader to infer the substitution. This is a refinement to N01's provenance design, not a separate defect count. [27]

The central invariant should be monotone evidence:

a run once invalidated by observed drift cannot become valid by restoration.

A completion flag, number of successful cases, source-stability flag, and runtime-exception status should remain independently inspectable. Avoid summarizing them as one ambiguous "passed" field.

12.2 Then close bounded adapter gaps with semantic laws

N05 has an unusually favorable cost-benefit ratio. The admitted rational affine class has a short proof and a complete constructive procedure; no general symbolic inequality solver is needed. Integrate the radius constructor with proper real-operand admission and distinct unsupported/inconclusive results. Retain the successful interval and clauses. This expands useful Mathics coverage without weakening any branch theorem.

For N06, first inventory all internal `Limit` consumers and make intended positive-local-coordinate directions explicit. Only then change the omitted direction policy. Test the adapter's contract independently of public branch selection, so that an explicitly one-sided branch test cannot accidentally be used to "verify" a two-sided default.

For N03, introduce a sparse fast path with a bounded dense fallback and connect it to consumers that only need nonzero coefficients. For N04, seek an upstream early-exit position primitive or implement only the demonstrably needed subset. Do not add an untested generic tree walker merely to make an adapter look feature-complete.

Acceptance law	Positive controls	Negative / boundary controls
Provenance invalidity is monotone	Stable inputs and normal protocol success.	Change-and-restore, persistent edits, changed suite, interrupted run.
Timeouts are finite positive durations	Small positive rational and decimal values.	NaN, infinities, zero, negative values, malformed text.
Sparse work follows support	Two monomials at widely separated degrees; cancellation.	Unexpanded dense products; nonpolynomial factors; budget exhaustion before expansion.
First-match search stops when justified	Early and late first match with matching order.	No match, root/head matches, negative levels, held default effects.
Affine germs survive shrinking	Arbitrarily small rational neighborhoods and positive coordinate changes.	Truly false local relation, non-real operands, unsupported symbolic signs.
Limit directions are preserved	Explicit left/right and continuous two-sided controls.	Jump, unresolved side, unequal sides, unsupported direction.

These laws are a more precise addition than another generic request for more unit tests. Their examples should be generated across scales and syntax forms, with expected values coming from elementary mathematics or an independent reference model. Runtime agreement is useful, but agreement between two paths sharing the same flawed helper is not independent evidence.

12.3 Profile the new adapters at the correct granularity

For N03, measure constructor time, maximum resident memory, and coefficient positions allocated while holding support fixed and increasing degree. Then hold degree fixed and increase true support. Record whether the sparse path, dense fallback, or refusal was selected. A benchmark that changes both support and degree cannot identify which cost was removed.

For N04, vary first-match position and total match count independently. Record traversed nodes and retained positions inside a controlled implementation, not only wall-clock time. Tiny cases can be dominated by interpreter startup; large held trees can be dominated by input construction. Measure those phases separately.

For N05, compare the number of exact sign checks and the available input class before comparing speed. The major improvement is that the method proves valid small neighborhoods at all; treating an unsupported proof as a fast successful computation would invert the performance conclusion.

The existing CI uses a 35-minute job deadline and a 300-second individual-case timeout. Those limits are not contradictory, but they bound different things: a sequence of several worst-case cases can exhaust a shard even when each case respects its own deadline. Use recorded per-case costs to balance grouped selections and retain partial results on deadline exhaustion. This is a consequence of the inspected configuration, not an observation of a failed CI run. [28]

No speedup factor is asserted in this article. The supplied evidence counts operations or demonstrates runner behavior; it does not measure the package under Mathics, and no honest Wolfram/Mathics speed ratio follows from it.

12.4 Handle the existing feature roadmap without claiming novelty

The repository’s larger targets remain valuable: full native input coverage, `DiscreteAsymptotic` delegation, and the broader vendored asymptotics program. The guide explicitly marks these as requirements rather than completed implementations. The older register already records integration, uniform parameter asymptotics, complex sectors, additional exponential grades, and coupled systems. Repeating that list as new missing features would not advance this audit. [4, 7]

The incremental recommendation is instead an implementation discipline for those targets. For each new family, record four separate outcomes: mathematical admission, symbolic construction, supported operations on the result, and numerical/certified evaluation. Make the runtime-specific adapter used at each stage visible in a diagnostic record. A feature can then be incomplete in Mathics at its numerical stage without falsely being called absent at its symbolic stage, or complete because one example printed successfully.

A native backend extension must also preserve native order, conditions, branch information, and unresolved results without silently promoting them to a custom analytic theorem. The current native contract separation provides an appropriate starting point; its already-recorded routing/default issues should remain owned by their existing work items. [10, 8]

13 Artifacts, reproduction, and interpretation of results

13.1 What is included

The archive contains the complete article source and compiled PDF, the exact upstream Python runner with its license, a guarded patch generator, a unified diff, the generated candidate runner, a separate Wolfram Language candidate package, diagnostic kernel probes, independent exact Python tests, and raw JSON records from the executed tests. The machine-readable novelty ledger links each new finding to its source and adjacent historical work items.

The Wolfram Language candidate file does not modify the user’s package or install replacements for `System` symbols. Its separate context is `AsymptoticReview`. It provides three narrow mechanisms: `SparseMonomialRules`, `AffineRadius`, and `ConservativeRealLimit`. Their admitted domains and unexecuted status are part of the deliverable, not omitted qualifications.

13.2 Re-running the executed experiments

From the archive’s root directory, the two Python programs reproduce the infrastructure and reference checks:

```
python -S tests/runner_experiments.py
python -S tests/exact_reference_checks.py
```

They use the standard library. The runner fixture was checked on Linux with Python 3.13.5 and uses a POSIX executable protocol peer; it is not claimed to be a Windows-tested fixture. Its temporary repository directories are isolated from the user’s checkout. The exact-reference script does not require a mathematical kernel.

The runner experiment has 14 scenarios: original and candidate variants for three source-stability cases and four invalid-timeout cases. Its overall assertions check *expected characterizations*, so some successful experiment assertions correspond to intentional failure exits of the tested runner. Do not read “assertions passed” as “the original implementation is correct.” Likewise, the exact reference cases are not new package test-suite passes.

13.3 Applying the candidate infrastructure patch

The patch generator refuses an unexpected upstream Git blob and refuses to write over its input. This prevents accidental patching of an evolved runner with a stale string-replacement recipe. A typical invocation is:

```
python -S code/patch_portable_runner.py \
  /path/to/checkout/validation/run_mathics_tests.py \
  /path/to/disposable/validation/run_mathics_tests.py \
  --diff /path/to/candidate.patch
```

The output belongs in a repository-shaped `validation` directory with `MathicsTests.wl` and the appropriate source tree. Running the bundled `code/run_mathics_tests_candidate.py` directly from its archive directory would give the original runner the wrong neighboring suite and repository root. The test fixture creates the necessary layout automatically.

The candidate is a reviewed repair proposal with focused Python evidence, not an upstream commit or a completed production integration. No repository writes were performed through the connector.

13.4 Running the pending kernel probes

Set `ASYMPTOTIC_REVIEW_SOURCE` to the absolute path of the pinned modular or standalone package entry. Optionally set `ASYMPTOTIC_REVIEW_CANDIDATES` to the absolute path of `code/ReviewPrimitives.wl`. Then run `tests/ProbeNewReview.wl` in a fresh mathematical kernel. Example launch forms, with executable locations adapted to the installation, are:

```
mathics --quiet --no-readline --file tests/ProbeNewReview.wl
WolframKernel -noinit -script tests/ProbeNewReview.wl
```

The script streams the package load before subsequent package expressions. Its observations must be read individually; it is not a self-declared acceptance suite. In particular, the tiny-domain public outcome in N05 must first be observed before being promoted from a source prediction to a runtime finding.

Run modular and standalone entries separately, retain complete messages and runtime versions, and distinguish original-source characterization from candidate-source acceptance. Successful primitive tests alone do not establish that every public consumer is correctly wired to a candidate. A Mathics proof failure may be a coverage gap, while an exception or inert expression may be an evaluator-compatibility defect; the report should not collapse those outcomes.

13.5 Limits of this audit

This is an incremental, evidence-bounded review, not a certification of the whole repository. The existing reviews were deduplicated through their indexes, complete consolidated finding crosswalks, and maintained implementation status; this article does not claim a full rereading of all 27 original articles. The current source inspection followed the new findings through selected consumers and examined the principal contracts and module map. No complete package execution, full-suite acceptance, end-to-end performance benchmark, or fresh interval-certificate validation was obtained.

The unavailable mathematical runtime prevents claiming a reproduced public Mathics result for N03–N06. It does not prevent proving the affine-radius lemma, demonstrating the adapter’s allocation mechanism, comparing documented limit contracts, or executing the actual Python infrastructure. Keeping these boundaries explicit makes the results usable rather than making them stronger than the evidence.

14 Conclusion

The strongest new defect is in evidence production: the portable runner can forget a detected source mismatch and issue a successful final receipt after restoration. A small, exercised patch makes invalidity cumulative and rejects nonfinite timeout inputs early. Those changes should be the first integration step.

The Mathics findings identify a productive second step. Sparse coefficient extraction and first-match search should not allocate work proportional to irrelevant degree gaps or all later matches. Affine eventual-domain proofs can be made independent of an arbitrary seven-radius grid using an exact constructive argument. Omitted limit directions should preserve the question being asked, while callers that intentionally use a positive local coordinate should remain explicitly one-sided.

The package’s mathematical value lies in its admitted real models, inverse branches, structured scales, and retained error information, not in a claim that native Wolfram Language lacks sophisticated asymptotics. Its Mathics layer is already substantial, but the correct unit of compatibility is a versioned operation with a preserved contract and corresponding evidence. New coverage should be built on those precise units, rather than inferred from loading, printed expressions, or historical pass totals.

A Source map and audit scope

All package references below are pinned to the review commit unless explicitly identified as historical or upstream Mathics sources. The included CSV source inventory gives the same paths and the purpose of inspection. Large modules were inspected in relevant ranges; listing a module is not a claim that every line or every public route was exhaustively checked.

Source / group	Use in this review
README.md; src/Documentation/User Guide.md	Public feature and coverage targets, order conventions, loading, native and Mathics boundaries. [3, 4]
docs/development/CODE_REVIEW_STATUS.md; WAVE_3_INTAKE.md	Deduplication, completed-fix baseline, complete attributed finding crosswalks, proposal ownership. [7, 8]
src/Kernel/AsymptoticAnalysis.wl; NativeCompatibility.wl	Entry, exactness and coordinate conventions, forward/native importer paths, object access, back-end separation. [9, 10]
InverseFunctionExpressions.wl; InverseFunctionBranches.wl	Eventual-condition admission, public consumers, branch validation, explicit limit direction. [11, 12]
LambertInverse.wl; FourierCoefficients.wl	Specialized scale examples, existing algorithms and consumer context. [13, 14]
MathicsCompatibility.wl; MathicsAlgebra.wl	Scoped control flow, associations, first-position search, dense coefficient-rule path. [17, 18]
MathicsAssumptions.wl; MathicsSimplification.wl	Realness/sign proof boundaries, simplification safety, local default limit. [19, 20]
MathicsInverseBranches.wl	Polynomial real-domain proof, affine interval test, seven-radius eventual search. [21]

Source / group	Use in this review
<code>MathicsCalls.wl</code> ; <code>MathicsTaylor.wl</code> ; <code>MathicsSpecialFunctions.wl</code> ; <code>MathicsCoreFunctions.wl</code> ; <code>MathicsTimeBudget.wl</code>	Held evaluation, bounded special-function coverage, numerical and timing boundaries. [22, 23, 24, 25, 26]
<code>validation/run_mathics_tests.py</code> ; <code>.github/workflows/mathics.yml</code>	Entire runner retrieved and exercised with a mock peer; workflow grouping and timeout configuration inspected. [27, 28]
<code>Mathics3 10.0.1 mathics/builtin/list/eol.py</code> ; <code>mathics/builtin/specialfns/expintegral.py</code>	Upstream implementation of bounded/unbounded position queries and Lambert numerical dispatch. [29, 30]

B Finding record format

The novelty ledger assigns each item a severity, evidence type, source mechanism, nearby historical owner, and explicit new contribution. The raw experiment records deliberately include unsuccessful original behavior, candidate behavior, and controls. The article and ledger do not relabel source-only predictions as executed kernel observations. Future updates should add runtime records and integration revisions rather than overwrite these historical evidence boundaries.

References

- [1] Source policy. Package links are pinned to the reviewed Git revision. Mathics implementation links use tag 10.0.1. Native interface claims use official documentation retrieved during the review. Paths in the archive refer to supplied artifacts, not external acceptance receipts.
- [2] Asymptotic contributors. [Commit7d1bc832895cc90a9b2a978b7b7684acab908bd2](#).
- [3] Asymptotic contributors. [README.md](#).
- [4] Asymptotic contributors. [src/Documentation/UserGuide.md](#).
- [5] Asymptotic contributors. [src/Kernel/README.md](#).
- [6] Asymptotic contributors. [external-reports/code-review/README.md](#).
- [7] Asymptotic contributors. [docs/development/CODE_REVIEW_STATUS.md](#).
- [8] Asymptotic contributors. [docs/development/WAVE_3_INTAKE.md](#).
- [9] Asymptotic contributors. [src/Kernel/AsymptoticAnalysis.wl](#).
- [10] Asymptotic contributors. [src/Kernel/NativeCompatibility.wl](#).
- [11] Asymptotic contributors. [src/Kernel/InverseFunctionExpressions.wl](#).
- [12] Asymptotic contributors. [src/Kernel/InverseFunctionBranches.wl](#).
- [13] Asymptotic contributors. [src/Kernel/LambertInverse.wl](#).
- [14] Asymptotic contributors. [src/Kernel/FourierCoefficients.wl](#).
- [15] Asymptotic contributors. [docs/Mathics/COMPATIBILITY.md](#).
- [16] Asymptotic contributors. [docs/Mathics/CALLABLES.md](#).

- [17] Asymptotic contributors. [src/Kernel/MathicsCompatibility.wl](#).
- [18] Asymptotic contributors. [src/Kernel/MathicsAlgebra.wl](#).
- [19] Asymptotic contributors. [src/Kernel/MathicsAssumptions.wl](#).
- [20] Asymptotic contributors. [src/Kernel/MathicsSimplification.wl](#).
- [21] Asymptotic contributors. [src/Kernel/MathicsInverseBranches.wl](#).
- [22] Asymptotic contributors. [src/Kernel/MathicsCalls.wl](#).
- [23] Asymptotic contributors. [src/Kernel/MathicsTaylor.wl](#).
- [24] Asymptotic contributors. [src/Kernel/MathicsSpecialFunctions.wl](#).
- [25] Asymptotic contributors. [src/Kernel/MathicsCoreFunctions.wl](#).
- [26] Asymptotic contributors. [src/Kernel/MathicsTimeBudget.wl](#).
- [27] Asymptotic contributors. [validation/run_mathics_tests.py](#).
- [28] Asymptotic contributors. [.github/workflows/mathics.yml](#).
- [29] Mathics3 project. Version 10.0.1, [mathics/builtin/list/eol.py](#).
- [30] Mathics3 project. Version 10.0.1, [mathics/builtin/specialfns/expintegral.py](#).
- [31] Wolfram Research. *Series*. Wolfram Language Documentation.
- [32] Wolfram Research. *SeriesData*. Wolfram Language Documentation.
- [33] Wolfram Research. *InverseSeries*. Wolfram Language Documentation.
- [34] Wolfram Research. *Asymptotic*. Wolfram Language Documentation.
- [35] Wolfram Research. *AsymptoticSolve*. Wolfram Language Documentation.
- [36] Wolfram Research. *DiscreteAsymptotic*. Wolfram Language Documentation.
- [37] Wolfram Research. *CoefficientList*. Wolfram Language Documentation.
- [38] Wolfram Research. *Position*. Wolfram Language Documentation.
- [39] Wolfram Research. *Limit*. Wolfram Language Documentation.
- [40] T. Bray, ed. *The JavaScript Object Notation (JSON) Data Interchange Format*. RFC 8259 (2017), Section 6: Numbers.